Lesson notes · for you

Algorithms and software engineering: unit check

Programming · Level 4 · Unit check

Time	Materials
20 min	Paper and a pen. No device needed.

Learning goals

- How work grows: big O
- Recursion
- Data structures
- Testing like an engineer

Activity: Activity: Code review circle

1. Warm-up (3 min): Write one idea from each of the four lessons in your own words.
2. Write (7 min): Write a small function from this unit (first_repeat, balanced, digit_sum or search) with five asserts, including edge cases.
3. Review (7 min): Pass your code to the next person. They add one test they think will break it, and one helpful comment.
4. Fix (3 min): Get your code back and make every test pass, the new one included.

Discussion

- Which idea from this unit will you use in your next project?
- Which question surprised you, and why?
- Where would big O matter in a real app you use?

Check yourself

- One habit I will keep as a programmer

Try it this week

Take one program you wrote this year. Find its big O, replace a slow part with the right data structure, and protect it with five asserts, including edge cases.

Worksheet

Algorithms and software engineering: unit check

Programming · Level 4 · Unit check

Name _____ Class _____ Date _____

1 **An O(n²) algorithm takes 1 second for 1,000 items. About how long for 2,000 items?**

◯ About 2 seconds ◯ About 4 seconds ◯ About 1 second

2 **Binary search on a sorted list is O(log n).**

◯ True ◯ False

3 **Write has_pair(values, target): True if two different items add up to target. It must stay fast for 20,000 values, so use a set: for each v, is target − v already seen?**

```
def has_pair(values, target):
    # one pass with a set of the values seen so far
    return False
```

Write your program:

4 **fact(4) should be 24, but this returns 0. Which line is wrong?**

Circle the line with the mistake. Write it correctly:

```
1  def fact(n):
2      if n == 1:
3          return 0
4      return n * fact(n - 1)
```

5 **Match each operation to what it costs or gives.**

x in a_list	• • • •	The oldest item
x in a_set	• • • •	The newest item
stack.pop()	• • • •	About one step
queue.pop(0)	• • • •	Checks one by one: O(n)